



Functions

Mahir Labib Dihan
Lecturer, CSE, BUET



Table of Contents

- | | |
|---------------------------------------|--|
| 1. Why Functions? | 5. Return Types and the return Statement |
| 2. Function Syntax and Structure | 6. Variable Scope and Lifetime |
| 3. Parameters and Arguments | 7. Practical Function Examples |
| 4. Call by Value vs Call by Reference | 8. Header Files in C |

Why Functions?



The Problem with Copy-Paste

Scenario: You need to calculate the area of a circle at 5 different places in your program.

Without Functions:

- Copy-paste the formula 5 times: $\text{area} = 3.14159 * r * r$;
- What if the formula is wrong? Fix it in all 5 places!
- What if the requirement changes? Modify all 5 places!
- Code becomes long, repetitive, and error-prone.

With Functions:

- Write the logic once in a function.
- Call the function wherever needed.
- Need to fix/change? Modify only the function!

Principle: DRY - Don't Repeat Yourself!

What is a Function?

Definition

A **function** is a self-contained block of code that performs a specific task. It can take input (parameters), process it, and optionally return a result.

Analogy: A Vending Machine

- **Input:** You put in money and press a button (parameters)
- **Process:** Machine selects the item (function body)
- **Output:** You get a snack (return value)

Benefits of Functions:

- **Modularity:** Break complex problems into smaller parts
- **Reusability:** Write once, use many times
- **Maintainability:** Easier to debug and update
- **Abstraction:** Hide complex details behind a simple interface

Functions You Already Know

Function	Purpose	Header File
<code>printf()</code>	Print formatted output	<code><stdio.h></code>
<code>scanf()</code>	Read formatted input	<code><stdio.h></code>
<code>sqrt()</code>	Square root	<code><math.h></code>
<code>pow()</code>	Power (x^y)	<code><math.h></code>
<code>strlen()</code>	String length	<code><string.h></code>
<code>main()</code>	Program entry point	(Required)

Key Insight: `main()` is itself a function! Every C program must have exactly one `main()` function

Function Syntax and Structure



Anatomy of a Function

General Syntax

```
return_type function_name (parameter_list) {  
    // Function body  
    // Local variable declarations  
    // Statements  
    return value; // Optional (required if return_type is not void)  
}
```

Components:

1. **Return Type:** What type of value the function gives back (int, float, void, etc.)
2. **Function Name:** Identifier following same rules as variables
3. **Parameter List:** Input values (can be empty)
4. **Function Body:** The actual code enclosed in {}
5. **Return Statement:** Value sent back to the caller

Example: A Simple Function

```
#include <stdio .h>
// Function definition
int add (int a, int b) {
    int sum = a + b;
    return sum;
}

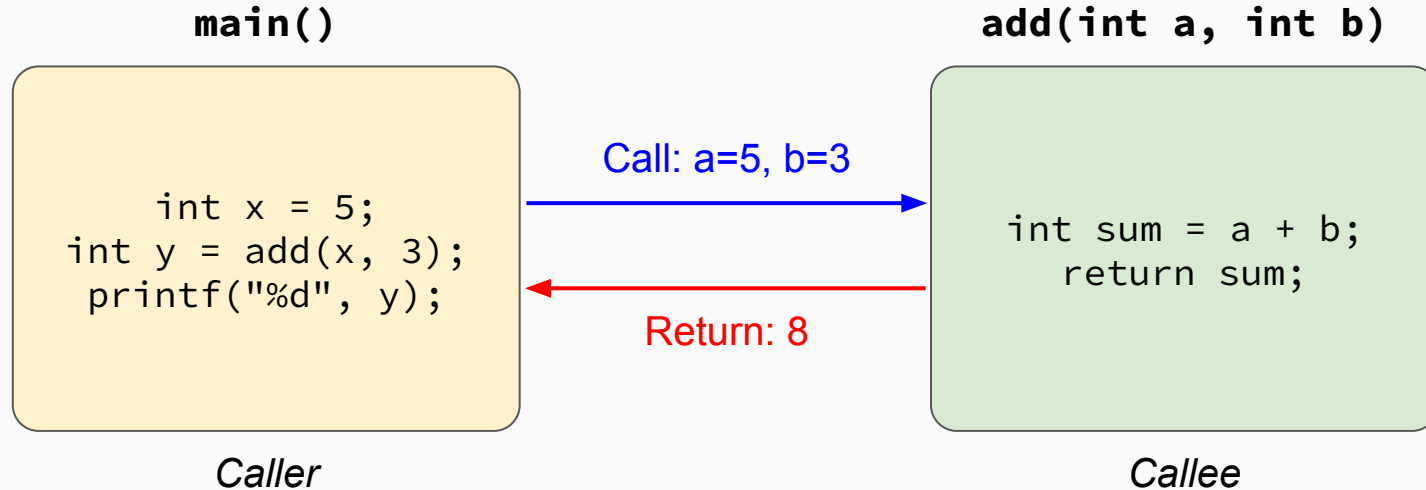
int main () {
    int result = add(5, 3); // Function call
    printf ("Sum = %d\n", result);
    // Can also use directly
    printf("Another sum = %d\n", add(10, 20));
    return 0;
}
```

Output:

Sum = 8

Another sum = 30

The Function Calling Process



Flow: `main()` → calls `add()` → `add()` executes → returns to `main()`

Function Declaration vs Definition

Declaration (Prototype):

- Tells the compiler the function exists
- Placed before `main()` or in header files
- Ends with a semicolon

```
// Declaration only  
int add (int a, int b);  
// Parameter names optional  
int add (int , int);
```

Definition:

- Contains the actual code
- Can be placed anywhere
- No semicolon after)

```
// Full definition  
int add (int a, int b) {  
    return a + b;  
}
```

Why Use Prototypes?

Allows you to define functions **after** `main()`, making code more readable (main at top).

Complete Example: Using Prototypes

```
#include <stdio.h>

// Function prototypes (declarations)
int add (int a, int b);
int multiply (int a, int b);

int main () {
    printf ("5 + 3 = %d\n", add(5, 3));
    printf ("5 * 3 = %d\n", multiply(5, 3));
    return 0;
}

// Function definitions (can be after main)
int add (int a, int b) {
    return a + b;
}
int multiply (int a, int b) {
    return a * b;
}
```

Parameters and Arguments



Parameters vs Arguments

Often confused but important distinction!

Parameters (Formal):

- Variables in function **definition**
- Act as placeholders
- Like variable declarations

```
int add(int a, int b)  
    a and b are parameters
```

Arguments (Actual):

- Values passed when **calling**
- The actual data
- Can be literals, variables, expressions

```
result = add(5, x+2);  
    5 and x+2 are arguments
```

Memory Analogy

Parameters are like empty boxes with labels. Arguments are the actual items you put in those boxes when calling the function.

Types of Arguments

Arguments can be:

```
int x = 10, y = 20;  
// 1. Literal values  
result = add(5, 3);  
// 2. Variables  
result = add(x, y);  
// 3. Expressions  
result = add(x + 1, y * 2);  
// 4. Other function calls  
result = add(add(1, 2), add(3, 4)); // add (3, 7) = 10  
// 5. Mixed  
result = add(x, 100) ;
```

Key Point: Arguments are evaluated **before** being passed to the function.

Multiple Parameters and Order

```
// Function with multiple parameters of different types
float calculateBMI (float weight, float height) {
    return weight / (height * height);
}

// Order matters!
float bmi1 = calculateBMI(70.0, 1.75); // Correct : weight =70 , height =1.75
float bmi2 = calculateBMI(1.75, 70.0); // WRONG : weight =1.75 , height =70!
```

Critical Rule

Arguments are matched to parameters **by position**, not by name!

- 1st argument → 1st parameter
- 2nd argument → 2nd parameter
- ... and so on

Call by Value vs Call by Reference



Two Ways to Pass Arguments

Call by Value:

- Function receives a copy of the argument
- Changes to parameter don't affect original
- Original data is protected
- C uses this by default!

Analogy: Giving someone a photocopy of your document. They can modify it, but your original is safe.

Call by Reference:

- Function receives the address of the argument
- Changes to parameter affect the original
- More efficient for large data
- Not directly in C, but can be emulated with pointers

Analogy: Telling someone your home address. They can come and rearrange your furniture!

Call by Value: The Swap Problem

```
#include <stdio.h>

void swap (int a, int b) {
    int temp = a;
    a = b;
    b = temp;
    printf("Inside swap: a=%d, b=%d\n", a, b);
}

int main () {
    int x = 5, y = 10;
    printf("Before swap: x=%d, y=%d\n", x, y);
    swap(x, y);
    printf("After swap: x=%d, y=%d\n", x, y);
    return 0;
}
```

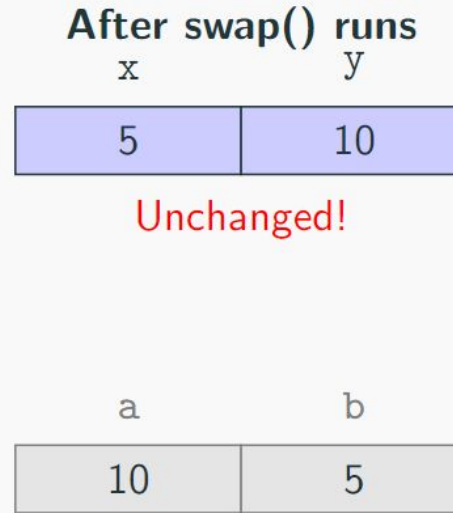
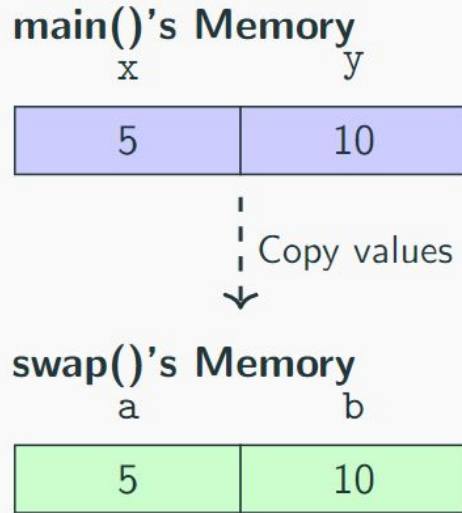
Output:

Before swap: x=5, y=10

Inside swap: a=10, b=5

After swap: x=5, y=10 ← x and y unchanged!

Memory Visualization: Why Swap Fails



Destroyed when swap() returns

Emulating Call by Reference with Pointers

The Idea: Instead of passing the value, pass the address (location in memory).

Key Concepts (Preview)

Arguments are matched to parameters **by position**, not by name!

- `&x` gives the **address** of variable `x`
- A **pointer** is a variable that stores an address
- `*ptr` accesses the value at the address stored in pointer `ptr`

Call by Value



Call by Reference



Swap That Actually Works (Using Pointers)

```
#include <stdio .h>
void swap (int *a, int *b) { // Receive addresses
    int temp = *a; // Get value at address a
    *a = *b; // Put value at b into address a
    *b = temp ; // Put temp into address b
}
int main () {
    int x = 5, y = 10;
    printf("Before: x=%d, y=%d\n", x, y);
    swap(&x, &y); // Pass addresses!
    printf("After: x=%d, y=%d\n", x, y);
    return 0;
}
```

Output:

Before: x=5, y=10

After: x=10, y=5 ← Actually swapped!

No need to learn pointers in detail. For now, just understand the concept.

Why scanf() Uses &

Now you understand why `scanf()` requires the `&` operator!

```
int age;  
scanf("%d", &age);
```

Explanation:

- `scanf()` needs to modify the variable `age`
- If we passed `age` by value, `scanf()` would only get a copy
- The copy would be modified, but `age` would remain unchanged!
- By passing `&age`, we give `scanf()` the address where to store the input

Remember

Any function that needs to modify its argument must receive the **address**, not the value. This is why C uses call by value by default – it's safer!

Return Types and the return Statement



Understanding Return Values

The return Statement

- Sends a value back to the calling function
- Immediately terminates the function execution
- The returned value replaces the function call in the expression

Return Types in C:

Type	Description
int	Returns an integer
float / double	Returns a decimal number
char	Returns a character
void	Returns nothing

Rule: The return type in the function definition must match the declared return type.

The void Return Type

Use void when a function performs an action but doesn't need to return a value.

```
void greet (char name []) {  
    printf ("Hello, %s!\n", name);  
    // No return statement needed (or use: return;)  
}  
  
void printLine () {  
    printf("=====\n");  
}  
  
int main () {  
    greet("Alice");  
    printLine();  
    // ERROR : Can't assign void to a variable  
    // int x = greet("Bob"); <- Compilation error!  
    return 0;  
}
```

Multiple Return Statements

A function can have multiple return statements, but only one will execute.

```
int absolute(int n) {  
    if (n >= 0) {  
        return n; // Exit point 1  
    } else {  
        return -n; // Exit point 2  
    }  
    // Code here is UNREACHABLE  
}  
  
char getGrade(int score) {  
    if (score >= 90) return 'A';  
    if (score >= 80) return 'B';  
    if (score >= 70) return 'C';  
    if (score >= 60) return 'D';  
    return 'F'; // Default case  
}
```

Return Type Mismatch

Question: What happens here?

```
int divide(int a, int b) { return a / b; }
float divideProperly(int a, int b) { return a / b; }

int main() {
    printf("divide(7, 2) = %d\n", divide(7, 2));
    printf("divideProperly(7, 2) = %f\n", divideProperly(7, 2));
    return 0;
}
```

Output:

```
divide(7, 2) = 3
divideProperly(7, 2) = 3.000000 <- Still 3, not 3.5!
```

Why? `a / b` performs integer division first ($7/2 = 3$), then converts to float.

Fix: `return (float)a / b;` or `return a / (float)b;`

Variable Scope and Lifetime



What is Scope?

Definition

Scope refers to the region of the program where a variable is visible and accessible.

Types of Scope in C:

1. **Local Scope:** Variables declared inside a function or block
2. **Global Scope:** Variables declared outside all functions
3. **Block Scope:** Variables declared inside {} blocks

Lifetime:

- **Local variables:** Created when function is called, destroyed when it returns
- **Global variables:** Exist for the entire program duration

Local vs Global Variables

```
#include <stdio.h>

int globalCount = 0; // Global: visible everywhere
void increment() {
    int localVar = 5; // Local : only in this function
    globalCount++; // Can access global
    printf("Local: %d, Global: %d\n", localVar, globalCount);
}
int main() {
    increment(); // Local: 5, Global: 1
    increment(); // Local: 5, Global: 2
    // printf("%d", localVar); // ERROR: localVar not visible
    printf("Final global: %d\n", globalCount); // 2
    return 0;
}
```

Warning

Global variables can be modified from anywhere! Use sparingly.

Shadowing: When Names Collide

Question: What is the output?

```
#include <stdio.h>
int x = 100; // Global x
void func() {
    int x = 50; // Local x shadows global x
    printf ("In func: x = %d\n", x);
}
int main() {
    int x = 25; // Local x in main
    printf("In main (before): x = %d\n", x);
    func();
    printf("In main (after): x = %d\n", x);
    return 0;
}
```

Output:

In main (before): x = 25

In func: x = 50

In main (after): x = 25

Shadowing: When Names Collide

What if you want a local variable to **remember** its value between function calls?

```
int counter() {  
    static int count = 0; // Initialized only ONCE at program start  
    count++;  
    return count ;  
}  
int main() {  
    printf("%d\n", counter()); // 1  
    printf("%d\n", counter()); // 2  
    printf("%d\n", counter()); // 3  
    return 0;  
}
```

Key Points:

- static variables are initialized only once
- They retain their value between function calls
- Useful for counters, caching, maintaining state

Practical Function Examples



Problem: Prime Number Checker

Task

Write a function that takes an integer as parameter and returns 1 if prime, 0 otherwise. Use it to print all primes from 2 to n.

Let's break this down:

1. First, understand what a prime number is
2. Design the isPrime() function
3. Optimize the function
4. Use the function to solve the complete problem

Step 1: Understanding Prime Numbers

Definition

A prime number is a natural number greater than 1 that has no positive divisors other than 1 and itself.

Examples:

- Prime: 2, 3, 5, 7, 11, 13, 17, 19, 23, ...
- Not Prime: 1 (by definition), 4 (2×2), 6 (2×3), 9 (3×3), ...

Key Observations:

- 2 is the only even prime number
- To check if n is prime, we need to verify no number from 2 to $n - 1$ divides n
- If n has a divisor d where $d > \sqrt{n}$, then $n/d < \sqrt{n}$ (so we only need to check up to $\sqrt{n}$)

Step 2: Naive isPrime() Function

Approach: Check if any number from 2 to $n-1$ divides n .

```
int isPrime(int n) {  
    // Handle edge cases  
    if (n <= 1) return 0; // 0 and 1 are not prime  
    // Check all numbers from 2 to n -1  
    for (int i = 2; i < n; i++) {  
        if (n % i == 0) {  
            return 0; // Found a divisor , not prime  
        }  
    }  
    return 1; // No divisor found , it 's prime  
}
```

Analysis:

- For `isPrime(100)`, we check: 2, 3, 4, 5, ..., 99 (98 checks!)
- For `isPrime(1000000)`, we do 999,998 checks!
- Can we do better?

Step 3: Optimization 1 – Check Only Up to $\sqrt{n}$

Key Insight: If $n = a \times b$ and $a \leq b$, then $a \leq \sqrt{n}$.

```
int isPrime(int n) {  
    if (n <= 1) return 0;  
    // Only check up to sqrt (n)  
    for (int i = 2; i * i <= n; i++) { // i * i <= n is same as i <= sqrt (n)  
        if (n % i == 0) {  
            return 0;  
        }  
    }  
    return 1;  
}
```

Improvement:

- For `isPrime(100)`, we check: 2, 3, ..., 10 (only 9 checks!)
- For `isPrime(1000000)`, we check up to 1000 (only 999 checks!)

Step 3: Optimization 2 – Skip Even Numbers

Key Insight: 2 is the only even prime. All other even numbers are not prime.

```
int isPrime(int n) {  
    if (n <= 1) return 0;  
    if (n > 2 && n % 2 == 0) return 0;  
    // Only check up to sqrt (n)  
    for (int i = 3; i * i <= n; i+=2) { // i * i <= n is same as i <= sqrt(n)  
        if (n % i == 0) {  
            return 0;  
        }  
    }  
    return 1;  
}
```

Further Improvement:

- We now check only half the numbers: 3, 5, 7, 9, ...
- Combined with $\sqrt{n}$ limit, this is very efficient!

Step 4: The Complete Program

```
#include <stdio.h>
int isPrime(int n) {
    if (n <= 1) return 0;
    if (n > 2 && n % 2 == 0) return 0;
    for (int i = 3; i * i <= n; i += 2) {
        if(n % i == 0) return 0;
    }
    return 1;
}
int main() {
    int n;
    printf("Enter n: ");
    scanf("%d", &n);
    printf("Prime numbers from 2 to %d:\n", n);
    for (int i = 2; i <= n; i++) {
        if (isPrime(i)) printf ("%d ", i);
    }
    printf("\n");
    return 0;
}
```


Example: GCD (Greatest Common Divisor)

Using the Euclidean algorithm: $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$ until $b = 0$

```
int gcd(int a, int b) {
    if (a < 0) a = -a; // Handle negatives
    if (b < 0) b = -b;

    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

int main() {
    printf("GCD(48, 18) = %d\n", gcd(48, 18) ); // 6
    printf("GCD(54, 24) = %d\n", gcd(54, 24) ); // 6
    return 0;
}
```

Example: Reverse an Integer

Reverse the digits of a number (e.g., 1234 $\rightarrow$ 4321)

```
int reverseNumber(int n) {
    int reversed = 0;
    int negative = (n < 0);
    if (negative) n = -n;
    while (n > 0) {
        int digit = n % 10;
        reversed = reversed * 10 + digit;
        n = n / 10;
    }
    return negative ? -reversed : reversed ;
}

int main() {
    printf("Reverse of 1234 = %d\n", reverseNumber(1234)); // 4321
    printf("Reverse of -567 = %d\n", reverseNumber(-567) ); // -765
    return 0;
}
```

Example: Count Digits

Count how many digits are in a number

```
int countDigits(int n) {
    if (n == 0) return 1; // Special case : 0 has 1 digit
    if (n < 0) n = -n; // Handle negative
    int count = 0;
    while (n > 0) {
        count++;
        n = n / 10;
    }
    return count;
}

int main() {
    printf("Digits in 12345 = %d\n", countDigits(12345)); // 5
    printf("Digits in 0 = %d\n", countDigits(0)); // 1
    printf("Digits in -99 = %d\n", countDigits(-99)); // 2
    return 0;
}
```

Header Files in C



What are Header Files?

Definition

Header files (.h files) contain function declarations, macros, and type definitions that can be shared across multiple source files.

Why Use Header Files?

- **Code Organization:** Separate interface from implementation
- **Reusability:** Share functions across multiple programs
- **Modularity:** Break large programs into manageable pieces
- **Standard Library:** Access pre-written functions

Two Types:

- `<header.h>` – System/standard headers (searched in system directories)
- `"header.h"` – User-defined headers (searched in current directory first)

Common Standard Header Files

Header	Common Functions
<code><stdio.h></code>	<code>printf</code> , <code>scanf</code> , <code>fopen</code> , <code>fclose</code>
<code><stdlib.h></code>	<code>malloc</code> , <code>free</code> , <code>atoi</code> , <code>rand</code> , <code>exit</code>
<code><string.h></code>	<code>strlen</code> , <code>strcpy</code> , <code>strcmp</code> , <code>strcat</code>
<code><math.h></code>	<code>sqrt</code> , <code>pow</code> , <code>sin</code> , <code>cos</code> , <code>log</code>
<code><ctype.h></code>	<code>isalpha</code> , <code>isdigit</code> , <code>toupper</code> , <code>tolower</code>
<code><time.h></code>	<code>time</code> , <code>clock</code> , <code>difftime</code>
<code><stdbool.h></code>	<code>bool</code> , <code>true</code> , <code>false</code> (C99)

Note for `<math.h>`

When using math functions, compile with `-lm` flag: `gcc program.c -lm`

Creating Your Own Header File

Step 1: Create the header file mymath.h

```
// mymath.h - Function declarations (prototypes)  
#ifndef MYMATH_H // Include guard: prevents double inclusion  
#define MYMATH_H  
int add(int a, int b);  
int multiply(int a, int b);  
int factorial(int n);  
#endif
```

Step 2: Create the implementation file mymath.c

```
// mymath.c - Function definitions  
#include "mymath.h"  
int add(int a, int b) { return a + b; }  
int multiply(int a, int b) { return a * b; }  
int factorial(int n) {  
    if (n <= 1) return 1;  
    return n * factorial (n - 1);  
}
```

Using Your Custom Header

Step 3: Use in your main program main.c

```
// main.c
#include <stdio.h>
#include "mymath.h" // Include YOUR header with quotes
int main () {
    printf ("5 + 3 = %d\n", add(5, 3));
    printf ("5 * 3 = %d\n", multiply(5, 3));
    printf ("5! = %d\n", factorial(5));
    return 0;
}
```

Step 4: Compile all files together

```
gcc main .c mymath .c -o program
./ program
```

The Flow: main.c includes mymath.h (declarations) → Linker connects to mymath.c (definitions)

Summary: Key Takeaways

1. **Functions** help organize code, enable reuse, and simplify maintenance
2. **Parameters** are placeholders; **arguments** are actual values passed
3. C uses **call by value** – functions get copies of arguments
4. **Call by reference** can be emulated using pointers (addresses)
5. The **return statement** sends a value back and exits the function
6. **Scope**: Variables are only visible in their declared region
7. **Header files** organize and share code across multiple files

Golden Rules

- One function = One task (Single Responsibility)
- Keep functions short and focused
- Use meaningful names for functions and parameters

Acknowledgement

- Mohammad Sadat Hossain, Lecturer, CSE, BUET